Lab 5 - Autonomous Navigation (A* planner + pure pursuit)

Aim. Give DiffBot a goal anywhere on the map. It plans a collision-free path around the walls and drives there on its own. You click a goal in RViz2 or send it from the command line.

Time. 90 min. **Needs.** Only this folder (no compiler).

This folder is a complete project

Everything this lab needs is in this folder. Nothing from the other labs is required.

Requirements: Windows 10/11, PowerShell, pixi (no administrator rights needed; install steps are in README-SETUP.md), and nothing else.

In this folder	
`ws\src\`	the ROS 2 packages; `ws\install\` appears after `pixi run build`
`scripts\`	helper scripts and the automated tests
`pixi.toml`	the environment and every `pixi run ...` command; `README.md` this manual; `tools\replay.ps1` re-runs the manual's commands as a self-test

Set it up once. `pixi install` puts ROS 2 Jazzy into `C:\rlabs` (a few GB; pixi's package cache makes the second lab on the same machine fast). Open PowerShell in this folder and run:

```
pixi install
pixi run verify
pixi run build
```

(`pixi run build` compiles this lab's packages into `ws\install`; run it again after every edit in `ws\src`.)

Run one lab at a time, and everything from this folder.

Theory

Navigation has three parts:

Part	Node	What it does
Global planner - *where* to go	`planner_server`	inflates the walls by the robot radius + margin (0.30 m), finds the shortest path with **A*** (8-connected, no corner cutting), shortens it ("string pulling"), resamples it every 0.1 m, publishes `/plan`
Controller - *how* to follow it	`pure_pursuit` (inside `navigator`)	steers towards the point 0.35 m ahead on the path; slows near the goal; turns on the spot when facing away
Executive - supervise	`navigator`	serves the **action** `navigate_to_pose`, asks the planner for a path (service `plan_path`), reads the robot pose from TF (`map -> base_footprint`), sends `/cmd_vel`, reports progress, aborts when stuck

An **action** is a service with feedback and cancel: the client sends a goal, gets `distance_remaining` while it runs and a result at the end. Result codes of this navigator: 0 success, 1 no plan, 2 no pose (TF), 3 blocked / stuck, 4 timeout.

Map (frame `map`, metres): room 10 x 8; wall 1 at x = 3.0-3.2 (y 0-5); wall 2 at x = 6.0-6.2 (y 3-8); a box at (4.4-5.2, 5.5-6.3); robot starts at (1, 1). Going from (1, 1) to (8.5, 6.5) therefore needs an "S": up around wall 1, down and around wall 2.

Procedure

1. T1 - start the whole stack (simulator, planner, navigator, RViz2). Cold start takes 30-40 s on Windows.

```
pixi run demo-nav
```

2. T2 - look at what is running. The navigator and planner are nodes; `plan_path` is the service between them; `interface show` shows the goal / result / feedback fields of the action; the planner's inflation is a parameter.

```
pixi run ros node list --no-daemon --spin-time 5
pixi run ros service list --spin-time 5
pixi run ros interface show nav2_msgs/action/NavigateToPose
pixi run ros param get /planner_server inflation_radius --no-daemon --spin-time 5
```

3. In RViz2 - click a goal. Choose the **Nav2 Goal** tool in the top toolbar and click a free spot, for example near the top-right of the room. The green line is the planned path; the robot follows it. (A click inside a wall is rejected.)

```
# a human clicks in the RViz2 window
```

4. T2 - the same from the command line. `goto` sends the action goal and prints progress every 2 s; the exit code is 0 for "reached". It takes about a minute.

```
pixi run goto 8.5 6.5
```

5. T2 - a goal inside a wall. (3.1, 2.0) is inside wall 1. The planner finds no path and the navigator aborts with code 1.

```
pixi run goto 3.1 2.0
```

6. T2 - check that nothing was hit. `/collision_count` publishes only on a collision, so 15 s of silence is the pass condition.

```
pixi run ros topic echo /collision_count --no-daemon --spin-time 5
```

7. Experiment: shrink the safety margin. The robot radius is 0.17 m. With only 0.22 m inflation there is just 5 cm of slack for tracking error. Replace the planner node by one with `inflation_radius:=0.22` and send the same far goal. Stop T1 first, then:

```
# Ctrl+C in T1
```

```
pixi run demo-nav
```

Close the planner of that launch and start your own (T3):

```
Get-Process planner_server | Stop-Process -Force
pixi run ros run diffbot_navigation planner_server --ros-args -p inflation_radius:=0.22
```

```
pixi run goto 8.5 6.5
```

```
pixi run ros topic echo /collision_count --no-daemon --spin-time 5
```

Record in the table whether it reached the goal and whether `/collision_count` published anything. (Stop all terminals afterwards.)

```
# Ctrl+C in T1 and T3
```

Observations (recorded)

Quantity	Value
`goto 8.5 6.5`	REACHED, last feedback 0.15-0.3 m from the goal, about 59 s, path about 13 m (straight line 9.3 m)
`goto 3.1 2.0` (inside wall)	`FAILED: no plan - planner returned no path`, exit code 1
`/collision_count`	no message (0 collisions)
`inflation_radius` 0.30 (default)	`Double value is: 0.3`
`inflation_radius:=0.22`	planner log: `inflated by 5 cells` (instead of 6); the goal was still **reached** (last feedback 0.25 m), no collision message. The margin is smaller, not yet broken

Verification

```
pixi run test-class5
```

9 checks: the action server comes up; a goal at (8.5, 6.5) succeeds; the robot ends within 0.25 m of it; `/plan` was published; the robot detours (travelled more than 1.15 x the straight line); no collision; a goal in a wall is aborted with code 1. Recorded: reached 0.14 m from the goal, 13.2 m driven, 0 collisions.

Questions

1. Why is the planner given a larger "robot" than the real one (0.30 m instead of 0.17 m)? Which error does the extra margin pay for?
2. Why does the navigator need TF (`map -> base_footprint`), and which two earlier labs produce that transform?
3. The planner and the navigator talk through a *service*, the client and the navigator through an *action*. Why is each the right choice?
4. Write your own `plan_path` service node (for example Dijkstra, or a straight line). The navigator needs no change - what property of ROS 2 makes that possible?

5. *(Extension)* Make the navigator honour the goal's orientation; re-plan every 2 s so the robot can cope with a new obstacle.

If it does not work

Symptom	Fix
`goto` waits at "waiting for the navigator"	T1 is not running or still starting (wait 40 s)
`goto` says "goal rejected"	the navigator is not active yet; wait and retry
RViz2 map is blank	set *Fixed Frame* to `map`; wait for the map
Robot does not move	`pixi run ros node list --no-daemon --spin-time 5` must show `/diffbot_sim`, `/navigator`, `/planner_server`